SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5101

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I Karnam Bhasakar Gnanesh(192324080) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
K B Gnanesh

Annexure A
Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

---

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Debugging Hash Function Implementation (SHA Family)

A. Identify errors in message padding and block processing steps.

Common errors in a SHA-based hashing implementation include incorrect message padding, wrong block sizes, incorrect length encoding, and improper handling of the final block. SHA-256 processes data in 512-bit blocks and appends a 64-bit representation of the original message length during padding. Incorrect padding can produce inconsistent or incorrect hash values.

B. Fix issues related to bitwise operations and endianness handling.

The implementation should use correct bitwise operations, including XOR, AND, NOT, right rotation, and logical right shifts. All intermediate values must be restricted to 32 bits using a mask such as 0xFFFFFFFF. SHA-256 uses big-endian byte order when interpreting message words and encoding the final digest.

A reliable solution is to use Python's built-in hashlib module:

import hashlibdef sha256_file(filename): sha = hashlib.sha256() with open(filename, "rb") as file: for chunk in iter(lambda: file.read(1024 * 1024), b""): sha.update(chunk) return sha.hexdigest()print(sha256_file("largefile.bin"))

C. Optimize the implementation for files of 50 MB or larger.

- Read files in chunks instead of loading the entire file into memory.
- Use the optimized native implementation provided by hashlib.
- Avoid unnecessary copying of data and repeated memory allocations.
- Process data incrementally using sha.update().

This approach reduces memory consumption and improves processing speed while producing a consistent hash for the same file contents.

D. Evaluate secure hash functions compared with basic checksums.

Cryptographic hash functions such as SHA-256 provide strong resistance to collisions and deliberate manipulation. Basic checksums, such as simple additive checksums, are primarily designed to detect accidental errors and can often be deliberately modified or forged. Therefore, SHA-256 is more appropriate for verifying the integrity of important files against malicious modification, provided the expected hash is obtained from a trusted source.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A. Debug certificate validation and key exchange issues.

Common causes of handshake failures include expired certificates, incorrect hostnames, untrusted certificate authorities, missing intermediate certificates, and incompatible TLS versions or cipher suites.

Key exchange problems may occur because of incorrect parameter selection, incompatible algorithms, or invalid public keys. The implementation should validate certificate chains, check expiration dates and hostnames, verify key exchange parameters, and use supported TLS versions.

B. Optimize the handshake process to reduce latency.

- Use TLS 1.3, which generally reduces the number of round trips required for a full handshake compared with TLS 1.2.
- Reuse persistent connections where appropriate.
- Avoid unnecessary certificate verification operations while preserving required validation.
- Use efficient cryptographic libraries and supported cipher suites.

C. Implement session resumption.

Session resumption allows a client and server to establish subsequent connections without repeating every step of a full handshake. In TLS 1.3, session tickets and pre-shared keys can support resumption.

A Python example using the ssl module is:

import sslimport socketcontext = ssl.create_default_context()context.minimum_version = ssl.TLSVersion.TLSv1_2with socket.create_connection(("example.com", 443)) as sock: with context.wrap_socket(sock, server_hostname="example.com") as tls: print("TLS version:", tls.version()) print("Cipher:", tls.cipher()) print("Session reused:", tls.session_reused)

For actual session resumption, the client must retain and reuse an eligible session, and the server must support it. The example demonstrates a secure TLS connection; it does not by itself guarantee session reuse.

D. Analyze performance versus security trade-offs.

TLS 1.3 and session resumption reduce connection latency and computational overhead. However, resumed sessions depend on securely managed session tickets or keys, and early data in TLS 1.3 can be vulnerable to replay in certain circumstances. Sensitive, non-idempotent transactions should not rely on 0-RTT early data without appropriate replay protections. Certificate validation must never be disabled merely to improve performance.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A. Identify issues in parameter selection and random number generation.

Invalid DSA signatures may result from incorrect domain parameters, an incorrect private key, improper modular arithmetic, or inconsistent message hashing. Another major issue is generating a random nonce k that is outside the required range or is reused across signatures.

For DSA, the domain parameters include primes p and q, where q divides $p-1$, and a suitable generator g. The private key x must satisfy $1 \leq x \leq q-1$.

B. Fix errors in signature generation and verification.

The DSA signing process is:

$$ r=(g^k \bmod p)\bmod q $$

$$ s=k^{-1}(H(m)+xr)\bmod q $$

Here, $H(m)$ is the message hash and k is a fresh, secret nonce. Both r and s must be nonzero.

During verification:

$$ w=s^{-1}\bmod q $$

$$ u_1=H(m)w\bmod q,\qquad u_2=rw\bmod q $$

$$ v=((g^{u_1}y^{u_2}\bmod p)\bmod q) $$

The signature is valid if $v=r$, provided the parameters, public key, and signature values are valid.

In practice, use a maintained cryptographic library rather than implementing DSA manually. DSA is a legacy algorithm and is not approved for generating new signatures under current NIST digital-signature guidance; use an approved modern alternative, such as ECDSA or EdDSA where supported.

C. Optimize modular arithmetic operations.

- Use Python's three-argument pow(base, exponent, modulus) for efficient modular exponentiation.
- Use an established library for modular inversion and signature operations.
- Validate parameters once during key setup rather than repeating expensive checks for every signature.
- Avoid unnecessary conversions between integers and byte strings.

D. Evaluate the impact of randomness quality.

The nonce \(k\) must be unpredictable, unique for each signature, and kept secret. Reusing or exposing it can reveal the private key. Weak randomness can therefore compromise every signature associated with that key. A cryptographically secure random number generator or a correctly implemented deterministic nonce scheme is essential.

4. Debugging Key Management System

A. Identify flaws in key storage.

Storing cryptographic keys in plaintext, hardcoding keys in source code, using weak encryption, and sharing keys through insecure channels are major security flaws. Attackers who obtain the keys may decrypt protected information, impersonate users, or generate fraudulent signatures.

B. Fix key lifecycle management issues.

A secure key management lifecycle includes:

1. Generation: Generate keys using a cryptographically secure random number generator.

2. Storage: Store keys in a protected key store, hardware security module (HSM), or managed key management service.

3. Distribution: Use authenticated and encrypted channels to distribute keys securely.

4. Rotation: Rotate keys according to a defined policy and immediately replace compromised keys.

5. Revocation: Revoke keys that are compromised, expired, or no longer authorized.

6. Destruction: Securely remove keys when they are no longer required, including relevant backups where feasible.

C. Optimize secure storage using encryption and access control.

Use envelope encryption, in which data is encrypted with a data-encryption key and that key is protected by a key-encryption key. Restrict access through role-based access control and least-privilege permissions. Audit key usage, separate key administrators from data users where appropriate, and avoid storing encryption keys alongside the data they protect.

For production systems, managed key management services or HSMs can reduce the risk of accidental key exposure.

D. Analyze how poor key management compromises security.

Even strong cryptographic algorithms cannot protect a system if their keys are exposed. A stolen encryption key can allow an attacker to decrypt stored data, while a compromised signing key can enable forged signatures. Poor rotation and revocation practices can also

allow attackers to retain access after a breach. Therefore, key management is as important as selecting a strong cryptographic algorithm.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A. Debug encryption/decryption synchronization issues.

Data corruption can occur when encryption and decryption use different keys, initialization vectors, nonce values, cipher modes, or packet boundaries. Other causes include incomplete reads, incorrect packet lengths, missing authentication checks, and incorrect handling of the final data block.

The system should use a well-defined authenticated encryption format, transmit required non-secret parameters safely, and verify every authenticated data unit before accepting its contents.

B. Fix packet handling and integrity verification errors.

- Include packet lengths and sequence information in a well-defined protocol.
- Detect incomplete transfers and handle network interruptions safely.
- Use authenticated encryption, such as AES-GCM or ChaCha20-Poly1305, to provide confidentiality and integrity.
- Never reuse a nonce with the same key for these encryption schemes.
- Verify authentication tags before releasing decrypted data.
- Use a final cryptographic hash to verify that the received file matches the expected file, where a trusted expected hash is available.

C. Optimize throughput for files of 100 MB or larger.

- Stream files in chunks instead of loading the entire file into memory.
- Use efficient native cryptographic libraries.
- Reuse authenticated connections rather than repeatedly establishing new ones.
- Select an appropriate chunk size based on performance testing.
- Use bounded buffering and, where supported by the protocol, controlled pipelining.
- Measure throughput, CPU usage, memory consumption, and error rates.

A basic chunked file-reading pattern is shown below:

```
def copy_file(source, destination):    with open(source, "rb") as src, open(destination, "wb") as dst:        while True:            chunk = src.read(1024 * 1024)  # 1 MB            if not chunk:                break            dst.write(chunk)copy_file("source.bin", "destination.bin")
```

This example demonstrates memory-efficient file copying only. It does not implement encryption or a secure transfer protocol. For actual secure transfers, use a standard SFTP implementation or a well-tested authenticated encryption protocol.

D. Compare the optimized system with insecure file transfer methods.

Feature	**Insecure transfer (e.g., plain FTP)**	**Optimized secure transfer**
Confidentiality	Data may be transmitted in plaintext	Encryption protects data
Integrity	Limited or absent cryptographic protection	Authentication detects unauthorized modification
Authentication	Credentials may be exposed	Secure authentication mechanisms
Large-file performance	Depends on implementation	Chunking and connection reuse improve efficiency
Reliability	Corruption may go undetected	Integrity checks help detect corruption
Security risks	Greater exposure to interception	Stronger protection when correctly configured

Conclusion: A properly implemented secure transfer system protects confidentiality and integrity while maintaining good performance through streaming, efficient cryptographic libraries, and connection reuse. Standard SFTP should be preferred over a custom protocol because it has established security mechanisms and mature implementations.

Overall Result

The five tasks demonstrate how debugging and optimization improve cryptographic reliability, security, and performance. Correct hashing, secure TLS handshakes, sound digital-signature operations, robust key management, and authenticated file transfers help

prevent data corruption, unauthorized access, and cryptographic attacks. Using well-tested cryptographic libraries and established protocols is generally safer than implementing cryptographic primitives from scratch.